CYBERDUNGEON – SYMBOLIC AI GAME MASTER FOR DYNAMIC TACTICAL COMBAT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT

CS23533 - Foundations of Artificial Intelligence

Submitted by

2116240701300 – MAHALAKSHMI.A

2116240701287– LATIKA SHIVANI.N



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "**CYBERDUNGEON – SYMBOLIC AI GAME MASTER FOR DYNAMIC TACTICAL COMBAT**" is the bonafide work of **"MAHALAKSHMI.A - 240701300"** and **"LATIKA SHIVANI.N - 240701287"** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
1	INTRODUCTION	1
2	LITERATURE REVIEW	3
3	PROPOSED SYSTEM	6
3.1	ARCHITECTURE DIAGRAM	8
3.2	ENEMY TURN PIPELINE	9
4	MODULES DESCRIPTION	10
4.1	LEVEL GENERATOR MODULE (CSP)	11
4.2	PATHFINDING MODULE (A*)	12
4.3	RULE ENGINE MODULE	13
4.4	GAME AND COMBAT MODULE	15
4.5	WEB INTERFACE AND WEBSITE MODULE	16
5	IMPLEMENTATION AND RESULTS	17
5.1	EXPERIMENTAL SETUP	17
5.2	RESULTS	18
6	CONCLUSION AND FUTURE WORK	24
7	REFERENCES	25

ABSTRACT

Most game opponents are either scripted so simply that they look foolish, or driven by learning methods whose decisions nobody can explain. This project presents CyberDungeon, a turn-based 5×5 dungeon game in which every decision, from the layout of a level to the move of an enemy, is made by a classical Foundations of Artificial Intelligence technique that can be followed by hand. A Constraint Satisfaction Problem (CSP) solver, using backtracking search, the minimum remaining values heuristic and forward checking, designs a new dungeon for every game. A symbolic rule engine with six priority-ordered IF–THEN rules decides what the enemy does in each turn, and A* search with the Manhattan distance heuristic plans its route around the walls whenever the rules call for a move or a retreat. The reasoning behind every enemy action is printed in plain English. The game is written in Python with Flask, and a companion website runs the same A* and CSP algorithms live in the browser. Tests showed that A* returned a route of the same length as breadth first search in all 1,810 solvable random cases while examining about 48% fewer cells, and that the CSP solver produced a valid dungeon with a reachable key and exit for each of 300 seeds, each in under 2 ms.

CHAPTER 1

INTRODUCTION

Video games are one of the oldest testing grounds for artificial intelligence, and the enemy in a game is usually the first piece of AI a player meets. Many modern systems get their behaviour from training on data, which makes them strong but hard to question: when the enemy does something surprising, nobody can say which rule or which number caused it. The classical topics of a Foundations of AI course, namely rule-based reasoning, search and constraint satisfaction, work the other way round. They are simple enough to be traced on paper, and a game is a good place to show that they still produce believable behaviour.

Problem statement. A game needs three things from its AI. It needs levels that are fair, because a level in which the key cannot be reached is a bug and not a challenge. It needs an enemy that decides sensibly, by fighting when the player is close and healing or running away when it is hurt. And it needs the enemy to move around walls, because walking in a straight line into a wall looks broken. Placing objects at random solves none of these, and a hand-written set of if-statements for movement does not scale. The problem addressed in this project is to build a small dungeon game in which the level design, the decision making and the movement are all produced by well-known AI algorithms, and in which every decision can be explained to the player.

Our approach. CyberDungeon is built as a pipeline of three symbolic AI techniques. First, the level is treated as a Constraint Satisfaction Problem: the player, enemy, key, exit, two health packs and four walls are variables, the 25 cells of the grid are their values, and design rules such as “the exit is at least six steps from the player” are constraints. A backtracking solver with

minimum remaining values (MRV) and forward checking finds an assignment, and A* is used inside the solver to check that the key and exit can really be reached. Second, the enemy's choice of action is made by a rule engine that checks six IF–THEN rules from the highest priority to the lowest and acts on the first one that matches. Third, when the chosen action is a move, A* search finds the shortest route around the walls. After every enemy turn the game shows which rule fired, why, and which cells A* examined.

Objectives. The objectives of the project are:

- to build a turn-based 5×5 dungeon game with movement, combat, health packs, a key and an exit;
- to generate every level with a CSP solver so that each dungeon is different and has a reachable key and exit;
- to drive the enemy with a priority-ordered, rule-based inference engine whose decisions are deterministic and traceable;
- to use A* search for the enemy's chase and retreat routes and for a hint for the player; and
- to show the reasoning on screen and to publish a website on which the same algorithms can be tried live.

Scope. The game is played on a single 5×5 grid with one enemy, and all movement is in the four straight directions, so distances are measured in Manhattan steps. No machine learning, external AI service or probabilistic model is used, by design. The project is a Flask application in Python; the website is plain HTML, CSS and JavaScript and contains JavaScript ports of A* and the CSP solver so that the demonstrations run in the browser without a server. The experiments in Chapter 5 were run on these ports.

CHAPTER 2

LITERATURE REVIEW

Russell and Norvig (2021) describe an intelligent agent as something that perceives its environment through sensors and acts on it to reach a goal, and they organise the field around the ways such an agent can decide: by search, by constraint satisfaction and by reasoning with knowledge. The enemy in CyberDungeon is a small agent of exactly this kind. Its percept is the board turned into a few facts, its decision is made by rules, and its movement is a search problem.

Rule-based systems. A rule-based or production system keeps its knowledge as IF–THEN rules and a simple engine that chooses which rule to apply. Buchanan and Shortliffe (1984) described this approach in the MYCIN experiments, and one of its main strengths was that the system could explain a conclusion by showing the rules that produced it. Game programmers have used the same idea for a long time. Millington and Funge (2009) list rule systems next to state machines and decision trees as standard tools for character behaviour, and A* as the standard method for movement. CyberDungeon follows this tradition and keeps the explanation facility: the matched rule and its reason are shown after every enemy turn.

Heuristic search. Dijkstra (1959) gave a method for shortest paths in a weighted graph by always settling the nearest unsettled vertex. Hart, Nilsson and Raphael (1968) added an estimate of the remaining cost and introduced A*, which expands the node with the lowest $f = g + h$, where g is the cost so far and h is the estimate. If h never overestimates the true cost (it is admissible), A* returns an optimal path. On a four-connected grid the Manhattan distance is such an estimate, so A* is optimal there and examines fewer cells than a blind search, which is the behaviour tested in Chapter 5.

Cormen et al. (2009) cover breadth first search, which finds the path with the fewest edges and is the natural baseline for comparison.

Constraint satisfaction. A CSP is defined by variables, the domains of values they may take, and constraints between them. Mackworth (1977) studied consistency in networks of relations, which is the basis for removing impossible values before searching. Haralick and Elliott (1980) compared backtracking variants and introduced forward checking, which removes from the domains of unassigned variables the values that conflict with the latest assignment, together with the "fail first" principle that is now called minimum remaining values: choose the variable with the fewest legal values left, so that dead ends appear early. Russell and Norvig (2021) present backtracking search with these two improvements as the standard way of solving a CSP, and this is the solver used here.

Procedural content generation and explainability. Togelius et al. (2011) survey searchbased generation of game content, and Smith and Mateas (2011) show that describing a design space declaratively, as constraints, lets a solver produce content that is guaranteed to meet the designer's rules. Yannakakis and Togelius (2018) place both within the wider study of AI and games. A separate motivation is explainability: Gunning et al. (2019) describe the need for AI systems whose decisions can be understood by the people who use them. A game whose enemy narrates its own reasoning is a small, concrete example of this idea. Table 2.1 compares the three techniques used in this project.

Table 2.1: Comparison of the AI techniques used in CyberDungeon

Technique	Used for	What it guarantees	Cost	Explainable?
Rule engine (priority-ordered IF–THEN rules)	Choosing the enemy's action	The first matching rule is always the one applied	One pass over R rules (R = 6)	Yes: the rule and reason are shown
A* search (Manhattan heuristic)	Chase and retreat routes, hint	Shortest route when one exists	Exponential in the worst case; tiny on 25 cells	Yes: examined cells are shown
CSP: backtracking + MRV + forward checking	Designing each dungeon	Every returned level satisfies all constraints	Exponential in the worst case; about 11 assignments here	Yes: constraints are written as rules

From this review we saw a gap that the project fills. Existing game AI is often presented either as a hidden collection of scripts or as a learned model, and procedural level generators rarely say why a level is fair. A small, open game in which the level is solved as a CSP, the enemy decides by readable rules and moves by A*, and everything is traced on screen, is both a working game and a clear teaching example of how the three techniques fit together.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is a web-based dungeon game with an explainable AI opponent. The player moves on a 5×5 grid, fights an enemy, collects health packs and the key, and reaches the exit. At the start of every game a CSP solver designs the dungeon. At the end of every player turn the enemy perceives the board, chooses an action with the rule engine, uses A* when it has to move, and the game shows the full reasoning on screen.

Before writing code, the two search-based parts were formulated in the standard way. Table 3.1 shows the enemy movement as a search problem solved by A*, and Table 3.2 shows the level design as a constraint satisfaction problem.

Table 3.1: Problem formulation for enemy movement (A*)

Component	Definition in this project
State	A cell (x, y) of the 5×5 grid
Initial state	The cell where the enemy stands
Actions	Move up, down, left or right to a neighbouring cell that is inside the grid and not a wall
Goal test	The current cell is the target cell (the player's cell when chasing, the safest cell when retreating)
Path cost	1 per step; while retreating, cells next to the player carry an extra cost
Heuristic h(n)	Manhattan distance to the target, which never overestimates on a fourconnected grid
Solution	The sequence of cells from the enemy to the target, plus the list of cells A* examined

Table 3.2: Problem formulation for level design (CSP)

Component	Definition in this project
Variables	player, enemy, key, exit, health1, health2, wall1, wall2, wall3, wall4 (10 variables)
Domains	Any of the 25 cells for every variable, except the player, who may start only in one of the 4 corners
Constraints	All ten variables are in different cells; distance rules between pairs of variables (listed in Section 4.1); a global rule that the key and exit are reachable
Solution	An assignment of a cell to every variable that satisfies all constraints

A game then runs in the following steps:

1. A new game starts and the CSP solver generates the dungeon from a seed.
2. The player takes a turn: moves, attacks, collects an item, or asks for an A* hint.
3. The enemy's turn begins. build_facts() converts the board into facts such as the distance to the player, the enemy's health percentage, its potions, and whether a retreat route exists.
4. The rule engine checks the rules from priority 1 downwards and selects the first one whose condition is true.
5. If the selected action is a move, A* plans the route; attack and heal need no search.
6. The action is carried out and the board is updated.
7. An explanation is produced with the rule, the reason, the route and the cells A* examined, and it is shown to the player.

3.1 ARCHITECTURE DIAGRAM

The system has four parts: input, logic, output and the game state, as shown in Figure 3.1.

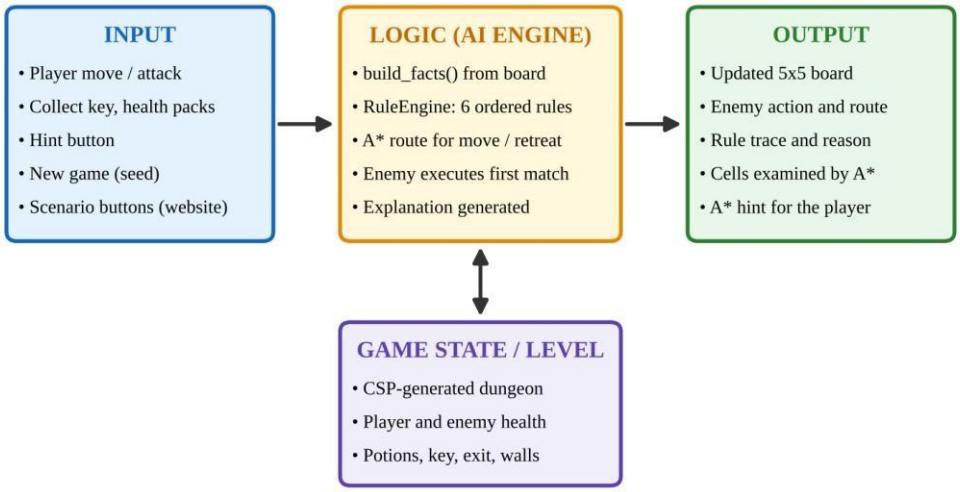


Figure 3.1: Architecture of CyberDungeon

Input. The player gives moves and attacks, and can ask for a hint. A new game can be started, and on the website the user can also press the scenario buttons to replay a rule trace.

Logic. This is the AI engine. It builds the facts, runs the rule engine, calls A* for movement, and prepares the explanation. The CSP solver belongs to the same layer but runs only once per game, when the level is created.

Game state. The generated dungeon, the health of both sides, the potions, the key, the exit and the walls are held here. The logic part reads from it and writes the result of every turn back to it, which is why the arrow between them points both ways.

Output. The updated board, the enemy's action and route, the rule trace with its reason, the cells that A* examined and the hint for the player are displayed.

3.2 ENEMY TURN PIPELINE

Figure 3.2 follows one enemy turn through the AI engine. The CSP solver has already created the level; the rest of the pipeline runs in every turn, and each stage hands a simple, readable result to the next.

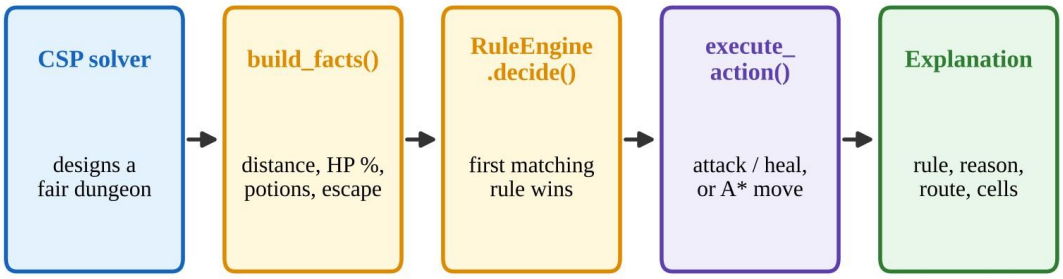


Figure 3.2: Pipeline of the AI engine in one enemy turn

CHAPTER 4

MODULES DESCRIPTION

The project is divided into modules with a single job each, which made the program easier to write, test and explain. Table 4.1 lists the source files and the section where each is described.

Table 4.1: Source files of the project

File	Purpose	Section
csp.py	Generic CSP solver: backtracking, MRV, forward checking	4.1
level_generator.py	Expresses the dungeon as a CSP and solves it	4.1
pathfinding.py	A* search on the grid	4.2
rules.py	Rule and RuleEngine classes, the symbolic core	4.3
ai_engine.py	Builds facts, calls the rule engine, uses A* for movement	4.3
player.py, enemy.py, combat.py	Player and enemy classes, attack and heal resolution	4.4
app.py	Flask routes and game state	4.4
templates/index.html, static/*	Game page, styles and the fetch-based turn loop	4.5
test_algorithms.py	Checks A* against BFS and re-verifies CSP levels	5.1
Website: index.html, style.css, script.js, algorithms.js	Landing page with live demos of the rule engine, A* and CSP	4.5

4.1 LEVEL GENERATOR MODULE (CSP)

This module designs the dungeon. csp.py contains a generic solver that knows nothing about games: it holds a set of variables, their domains, and binary constraints between pairs of variables, plus global checks that are judged once everything is placed. level_generator.py then states the dungeon as a problem for this solver. The constraints are:

- all ten objects are in different cells (an all-different constraint);
- the exit is at least 6 steps from the player, so the player has to travel;
- the enemy starts at least 3 steps from the player but within 2 steps of the key, so it guards the key without attacking at once;
- the key is at least 3 steps from the exit;
- each health pack and each wall is at least 2 steps from the player, and the two health packs are at least 3 steps apart; and
- a global rule that the player can reach the key, the key can reach the exit, and the enemy can reach the player, each checked with A* while the walls are in place.

Figure 4.1 shows the pairwise constraints as a graph. The solver works in three steps. It selects the unassigned variable with the fewest legal values left (MRV), breaking ties by choosing the variable that has more constraints. It tries the values of that variable in an order shuffled by a seeded random number generator, so the same seed always gives the same dungeon. After each assignment, forward checking removes from the neighbours' domains every value that now breaks a constraint, and if a domain becomes empty the solver backs up at once instead of going deeper. To avoid exploring the same level many times, symmetry breaking is added: the four walls, and the two

health packs, are interchangeable, so they are forced into increasing cell order. The core of the solver is:

```
def solve(variables, domains, constraints):
var = min(unassigned, key=lambda v:
len(domains[v])) # MRV for value in
shuffled(domains[var], rng): #
seeded order assignment[var] = value
pruned = forward_check(var, value, domains)
# drop conflicts if pruned is not None:
# no empty domain result = solve(...)
if result: return result del
assignment[var] #
backtrack return None
```

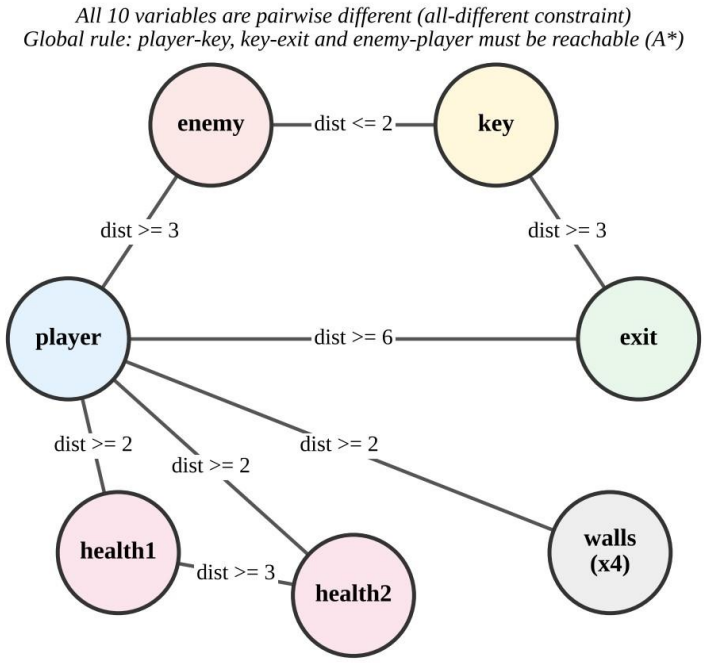


Figure 4.1: Constraint graph of the dungeon-design problem

4.2 PATHFINDING MODULE (A*)

pathfinding.py implements A* on the grid. Each cell has a cost g (steps from the start) and a score f = g + h, where h is the Manhattan distance to the goal. An open list holds the cells waiting to be expanded, and the cell with the lowest f is always taken first; a counter breaks ties in first-in, first-out order so that the result is repeatable. A closed set records the cells already expanded, and a

parent record allows the final route to be read back from the goal. Walls and cells outside the grid are never entered. Besides the route, the function returns the list of examined cells in order, which is what the game and the website show as “cells examined”.

```
while open:
    cell   =   pop_lowest_f(open)
# f = g + h    if cell in closed:
continue
    closed.add(cell);
explored.append(cell)      if cell ==
goal: return rebuild(came_from, cell),
explored               for   nxt   in
four_neighbours(cell):
        if outside(nxt) or nxt in walls:
continue            g_new = g[cell] + 1
if g_new < g.get(nxt, inf):        # a
cheaper way was found             g[nxt]
= g_new; came_from[nxt] = cell
            push(open,   f   =   g_new   +
manhattan(nxt, goal), nxt) return None,
explored                        # no route
exists
```

A* is used in three places: for the enemy’s chase, for its retreat (where the cells next to the player carry an extra cost so that it runs away without passing the player), and for the hint shown to the player. Because every step costs at least 1, the Manhattan distance never overestimates, so the route returned is always a shortest one. A* is also called by the level generator to prove that a level can be completed.

4.3 RULE ENGINE MODULE

This is the symbolic core of the project. ai_engine.py first calls build_facts(), which turns the board into plain facts: the distance to the player, whether the player is adjacent, the enemy’s health as a percentage, its number of potions, and whether a retreat route exists (found by A*). rules.py holds a Rule class,

with a name, a condition, an action and a reason, and a RuleEngine that tests the rules from the highest priority downwards and returns the first one that matches. Because the order of the rules decides what happens when several conditions are true, the order is the enemy's personality: it protects itself first, then fights, then moves. Table 4.2 lists the rules and Figure 4.2 shows the decision flow.

Table 4.2: The rulebook of the enemy

Priority	IF (condition)	THEN (action)	Reason shown to the player
1	health ≤ 30% and a potion is available	Heal	"My health is critically low and I still have a potion, healing now."
2	health ≤ 30%, no potion, a retreat route exists	Retreat (A* route)	"My health is critical and I'm out of potions, I must fall back!"
3	health ≤ 30%, no potion, cornered, player adjacent	Last stand (attack)	"I'm hurt, out of potions and cornered. Nowhere to run, so I fight!"
4	player is adjacent	Attack	"Player is right next to me, I strike while I can!"
5	player is far away	Move closer (A* route)	"Player is too far away, closing the distance."
Default	nothing else matches	Idle	"Nothing urgent to do right now, staying alert."

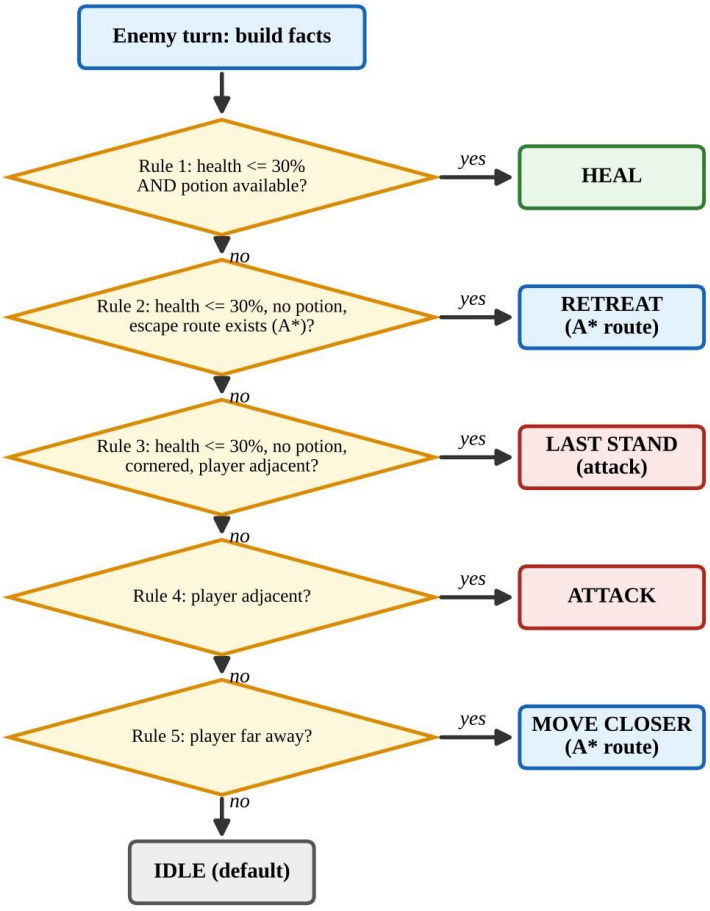


Figure 4.2: Decision flow of the rule engine

The engine is deterministic: the same facts always give the same action. This is what makes the enemy testable and its behaviour explainable, and it is the reason no probability or learning is needed.

4.4 GAME AND COMBAT MODULE

player.py and enemy.py define the two characters with their position and health, and the enemy also carries its potions. combat.py resolves attacks and healing so that all changes of health happen in one place. app.py is the Flask application: it creates a new game by calling the level generator, keeps the game state, and exposes routes that the browser calls for the player's move, the enemy's reply and the hint. The enemy's reply returns the chosen action together with the rule that fired, its reason, the planned route and the cells that A* examined, so that the interface can display them. The code is written in an

object-oriented way, with one class for each concept (Player, Enemy, Rule, RuleEngine, AStarPathfinder, CSP) and short, commented methods.

4.5 WEB INTERFACE AND WEBSITE MODULE

The game page is built with HTML and CSS and is driven by plain JavaScript that uses fetch requests to talk to Flask, so no front-end framework is needed. The board is drawn as a grid with overlays for the enemy's route and the cells that A* examined, and the reasoning is shown in a panel beside it.

A separate landing website explains the project and contains three live demonstrations. A **rule-trace console** replays five scenarios (player adjacent, health low with a potion, health low without a potion, cornered, and player far away) line by line, showing the facts, each rule that did or did not match, and the final action. An **A* demo** lets the visitor click cells to add or remove walls and then runs the real algorithm, colouring the cells it examines and the final route. A **CSP demo** solves a new dungeon in the browser and prints the number of assignments tried, backtracks, values pruned and the solving time. These demos use algorithms.js, a port of the Python algorithms, so what is shown on the website is the same logic as in the game. The same file can be run in Node.js, which allowed the experiments of Chapter 5 to be automated.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The game was written in Python with Flask, and the website in HTML, CSS and JavaScript. The experiments below were run on the JavaScript ports of A* and the CSP solver (algorithms.js), executed with Node.js, because these are the same algorithms in a form that can be called from a script. Breadth first search, used only as a reference in the tests, was written separately for the test script. Table 5.1 lists the setup.

Table 5.1: Experimental setup

Item	Details
Languages	Python 3 (game and Flask server); JavaScript (website and experiments)
Framework	Flask for the server; plain HTML, CSS and JavaScript for the interface
Test runtime	Node.js v22 running algorithms.js
AI methods	Rule engine, A* search, CSP (backtracking, MRV, forward checking). No machine learning or external AI services
Grid	5×5 cells, four-directional movement, Manhattan distance
Hardware	An ordinary computer; no GPU or special hardware

Four kinds of tests were planned. First, the rule engine was tested with five scenarios that cover the rules. Second, A* was run on seven hand-made wall layouts and compared with breadth first search (BFS). Third, 2,000 random wall layouts were used to compare A* and BFS more widely. Fourth, the CSP solver was run for 300 different seeds, and every solution was checked again against all the constraints by an independent function.

5.2 RESULTS

Test 1: Rule engine

Table 5.2 shows the five scenarios and the rule that fired in each. In every case the engine picked the first matching rule and nothing below it, and the reason printed was the one written for that rule.

Table 5.2: Rule engine test scenarios

Scenario	Facts	Rule that fired	Action
Player adjacent	adjacent = True, health = 100%	Rule 4	Attack
Low HP with potion	health = 22%, potions = 1	Rule 1	Heal (even with the player adjacent)
Low HP, no potion	health = 18%, potions = 0, can_retreat = True	Rule 2	Retreat along an A* route
Cornered	health = 18%, potions = 0, can_retreat = False, adjacent = True	Rule 3	Last stand (attack)
Player far away	distance = 3, adjacent = False	Rule 5	Move closer along an A* route

The trace printed for the cornered scenario is shown below. It shows why the enemy that is hurt, out of potions and unable to escape still fights, which is the behaviour the third rule was written to produce.

```
facts.enemy_health_percent       =       0.18,
enemy_potions = 0
A* found no escape: facts.can_retreat = False,
is_adjacent = True
-> rule 1  heal_if_low_and_has_potion ........
no match
-> rule 2  retreat_if_low_and_no_potion ......
no match
```

```
-> rule 3  last_stand_if_cornered ..........
MATCH
"I'm hurt, out of potions and cornered. Nowhere
to run, so I fight!" action: attack
```

Test 2: A* on seven layouts

In each layout the enemy stands in the first column and the player in the last. Table 5.3 gives the length of the route found by A*, the number of cells A* examined, and the number examined by BFS, which has no heuristic. In all five solvable layouts the A* route had the same number of steps as the BFS route, which is the shortest possible. In the two layouts with no route, A* and BFS both reported that no route exists after examining the same cells. **Table 5.3: A* compared with BFS on seven layouts**

No.	Layout	Walls	Route (steps)	Same as BFS	A* cells	BFS cells
1	Open grid	0	4	Yes	5	19
2	Default demo walls	2	6	Yes	11	21
3	Vertical barrier with a gap	4	8	Yes	17	18
4	Zig-zag corridor	6	12	Yes	16	19
5	Diagonal, corner to corner	0	8	Yes	25	25
6	Player boxed in	5	No route	Yes (both fail)	19	19
7	Full wall across the grid	5	No route	Yes (both fail)	10	10

Layout 1, the open grid, shows the benefit of the heuristic most clearly: A* walks straight to the goal and examines only 5 cells, while BFS spreads out in all directions and examines 19 (Figure 5.1). Layout 5 shows the opposite case: when the goal is in the far corner, the heuristic cannot narrow the search and both methods examine all 25 cells. Over the five solvable layouts A* examined 74 cells against 102 for BFS, which is 27% fewer. Figure 5.2 shows two of the

A* runs with the order in which cells were examined, and Figure 5.3 compares the two methods on all seven layouts.

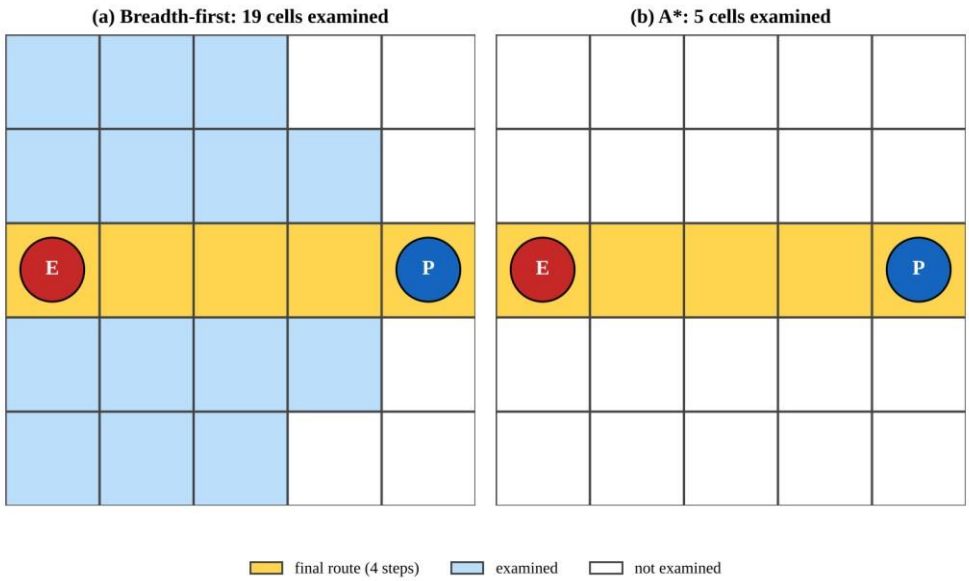


Figure 5.1: BFS and A* on the open grid (both find a 4-step route)

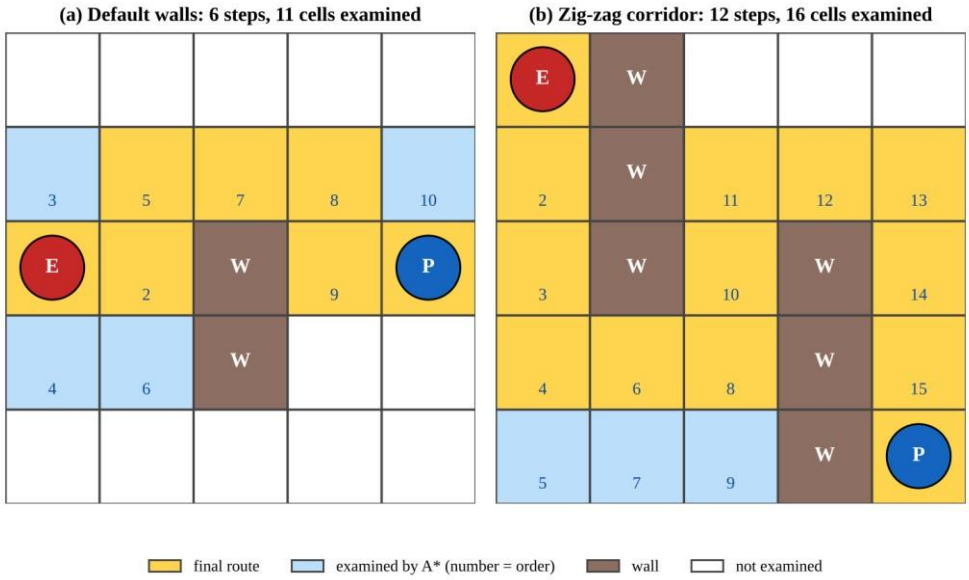


Figure 5.2: A* on the default walls and the zig-zag corridor

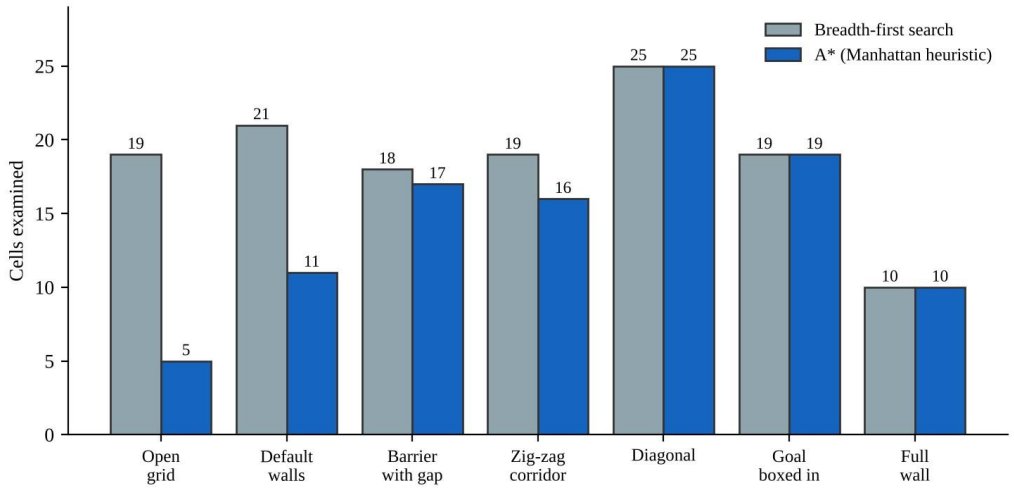


Figure 5.3: Cells examined by BFS and A* on the seven layouts

Test 3: A* on 2,000 random layouts

To check that these results were not special to the chosen layouts, 2,000 random layouts were generated, each with between 3 and 8 walls placed at random, and A* and BFS were run on each. Table 5.4 shows that the two methods agreed on whether a route exists in all 2,000 cases, and that whenever a route existed the A* route had exactly the length of the BFS route. This is the result expected from the theory of A* with an admissible heuristic, and it also matches the check done by test_algorithms.py in the game.

Table 5.4: A* compared with BFS on 2,000 random layouts

Measure	Result
Layouts tested (3 to 8 random walls each)	2,000
A* and BFS agree on whether a route exists	2,000 of 2,000
Layouts with a route	1,810 (90.5%)
A* route length equal to BFS route length	1,810 of 1,810
Average cells examined by BFS	15.93
Average cells examined by A*	8.33
Reduction in cells examined	47.7%

Test 4: CSP level generation

The CSP solver was run for seeds 1 to 300. All 300 runs returned a solution, and when every solution was re-checked against the distinct-cell rule, all the distance rules and the A* reachability rule, none of them broke a constraint. The 300 dungeons were all different from each other, and all four corners were used as the player's start (88, 73, 65 and 74 times). Figure 5.4 shows four of the generated dungeons together with the work the solver needed for each.

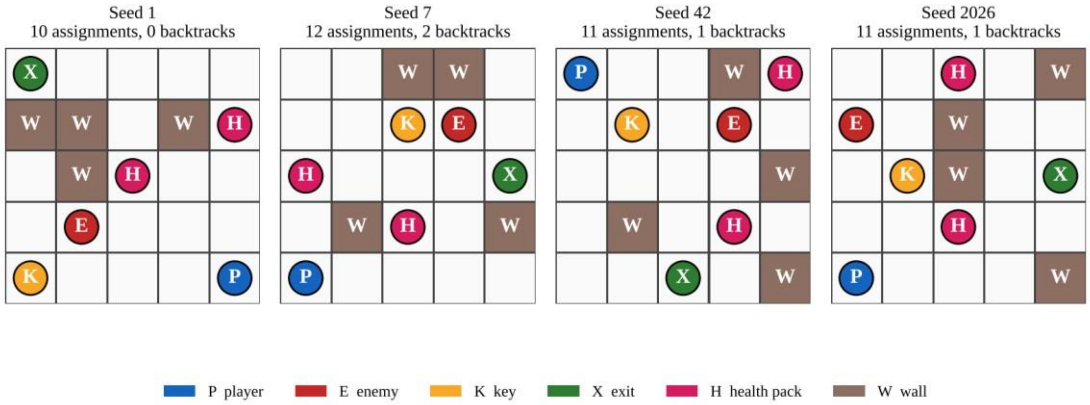


Figure 5.4: Dungeons generated by the CSP solver for four seeds

Table 5.5 summarises the effort. Ten assignments are the minimum possible, one for each variable, and the median run needed exactly that. Forward checking removed on average about 148 impossible values per dungeon before they could be tried, which is why the solver almost never had to back up: in 166 of the 300 runs (55%) there was no backtracking at all, and the worst case needed 34 backtracks. Every dungeon was solved in under 2 ms. **Table 5.5: Effort of the CSP solver over 300 seeds**

Measure	Minimum	Median	Mean	Maximum
Assignments tried	10	10	11.0	30
Backtracks	0	0	1.3	34
Values pruned by forward checking	120	140	147.6	230
Solve time (ms)	0.09	0.14	0.18	1.78

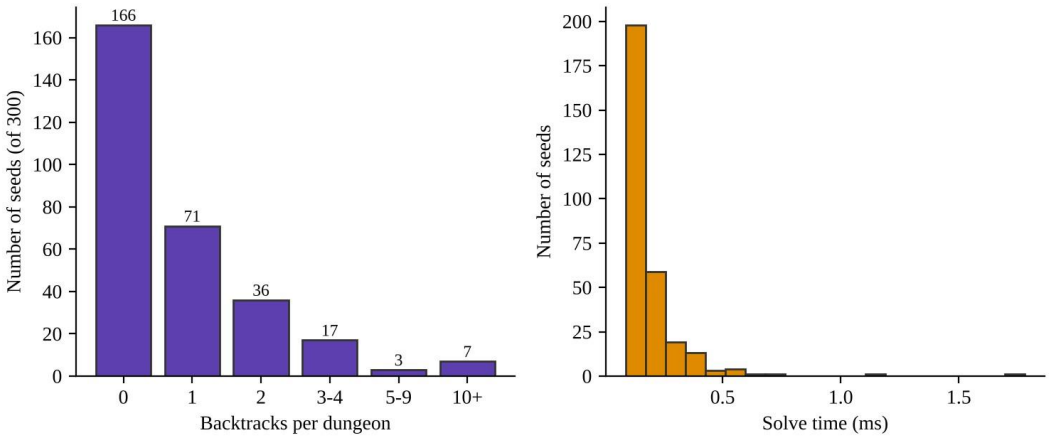


Figure 5.5: Backtracks and solving time over 300 seeds

The main observations from the tests are:

- The rule engine always applied the first matching rule, so the enemy's action was predictable and each action came with a readable reason.
- A* returned a route as short as BFS in every solvable case (1,810 of 1,810 random layouts and 5 of 5 hand-made ones).
- A* examined about 48% fewer cells than BFS on random layouts, with the largest saving when the goal lies straight ahead and no saving when the goal is in the far corner.
- When every route was blocked, the search ended cleanly and reported that no route exists instead of failing.
- The CSP solver produced a valid and different dungeon for every one of 300 seeds, mostly without backtracking, in under 2 ms each.

CHAPTER 6

CONCLUSION AND FUTURE WORK

In this project we built CyberDungeon, a turn-based dungeon game in which three classical AI techniques work together. A CSP solver designs each level, a priority-ordered rule engine chooses the enemy's action, and A* search plans its movement. Every decision is explained on screen in plain English, and a companion website lets anyone try the algorithms live. The tests showed that the rule engine fires the intended rule in each scenario, that A* gives routes as short as breadth first search while examining about half as many cells on random layouts, and that the CSP solver creates a valid, different dungeon for every seed in a few milliseconds at most.

Working on it helped us understand how a design problem is turned into variables, domains and constraints, and how small improvements such as MRV and forward checking change the search from trial and error into something that almost never backtracks. We also saw that the order of the rules is a design decision of its own: the same six rules in a different order would give the enemy a different personality.

The system has some limits. The game has one enemy on a small 5×5 grid. The distance rules in the CSP use Manhattan distance, which is simple but ignores walls, so the A* reachability rule has to be added to make sure a level can be finished. Because the rules are fixed, an experienced player can learn the enemy's behaviour. In future the work can be extended in the following ways:

- add several enemies with different rule sets, such as a cautious guard and an aggressive hunter;
- use the CSP solver to set difficulty, for example by changing the distance limits or the number of walls;

- try larger grids and compare A* with other searches such as Dijkstra's algorithm or bidirectional search;
- add a learning agent as an opponent and compare it with the rule-based enemy in win rate and in how well each can be explained;
- run a user study to measure whether the on-screen reasoning helps players understand and enjoy the game; and
- deploy the Flask game online and connect the "Play" buttons of the website to it.

REFERENCES

Buchanan, B.G. and Shortliffe, E.H. (1984) Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project, Addison-Wesley, Reading, MA.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) Introduction to Algorithms, 3rd edn, MIT Press, Cambridge, MA.

Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', Numerische Mathematik, Vol.1, No.1, pp.269-271.

Gunning, D., Stefik, M., Choi, J., Miller, T., Stumpf, S. and Yang, G.-Z. (2019) 'XAI— Explainable artificial intelligence', Science Robotics, Vol.4, No.37, eaay7120.

Haralick, R.M. and Elliott, G.L. (1980) 'Increasing tree search efficiency for constraint satisfaction problems', Artificial Intelligence, Vol.14, No.3, pp.263-313.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, Vol.4, No.2, pp.100-107.

Mackworth, A.K. (1977) 'Consistency in networks of relations', Artificial Intelligence, Vol.8, No.1, pp.99-118.

Millington, I. and Funge, J. (2009) Artificial Intelligence for Games, 2nd edn, CRC Press, Boca Raton, FL.

Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edn, Pearson, Hoboken, NJ.

Smith, A.M. and Mateas, M. (2011) 'Answer set programming for procedural content generation: A design space approach', IEEE Transactions on Computational Intelligence and AI in Games, Vol.3, No.3, pp.187-200.

Togelius, J., Yannakakis, G.N., Stanley, K.O. and Browne, C. (2011) 'Search-based procedural content generation: A taxonomy and survey', IEEE Transactions on Computational Intelligence and AI in Games, Vol.3, No.3, pp.172-186.

Yannakakis, G.N. and Togelius, J. (2018) Artificial Intelligence and Games, Springer, Cham.